

第 6 次组会汇报

效率与外部方法对比

对应工作：126、127 号实验；论文当前版本

本次要点

1. 统一计时（同一张空闲 GPU、同一批集合）：最终估计器每个集合 48—64 ms，比串行重训快 241—906 倍；读出改为数学等价的快速实现，结果逐位相同，快约 2.8 倍。

2. 与外部“免重训”方法对比：均值代替、LazyVI、热启动、掩码代理模型直接输出、TabPFN v2。只有 TabPFN 的精度与本文相当，但每集合慢 13—110 倍；认证之后两者恢复的风险相同。

3. 如实说明：TabPFN 的优势集中在训练行最少的 PJM，本文在那里有 0.03—0.04 的系统低估；它有时比重训的攻击器族还强，所以更适合作为攻击器族的一员。

1 统一计时（126 号）

1.1 先澄清“三种子”

表 1: 各方法每个查询集合的计算量

方法	每个查询集合的计算	读出次数	特征维度
旧 E（只预测目标 $\times 3$ ）	三个主干各自读出，预测取平均	3	300—326
C2（重建 $\oplus$ 随机 $\times 1$ ）	两个主干特征拼接，读出一次	1	556—582
本文 C3（重建 $\oplus$ 随机 $\times 3$ ）	C2 做三遍，预测取平均	3	556—582
三种子特征拼接（未采用）	三个主干特征拼成一块，读出一次	1	812—838

论文里的“ $\times 3$ ”都是三次独立读出再平均，而不是把三个种子的特征拼在一起读出一次。后者反而更慢（41 ms 对 21 ms），认证也更差（0.916 对 0.953）。

1.2 每个集合的耗时

图1 各方法每集合耗时：读出耗时随特征维度增长； $\times 3$ = 三次独立读出再平均；快速实现（全体 Gram 一次、各折做减法、 λ 批量求解）约快 2.8 倍

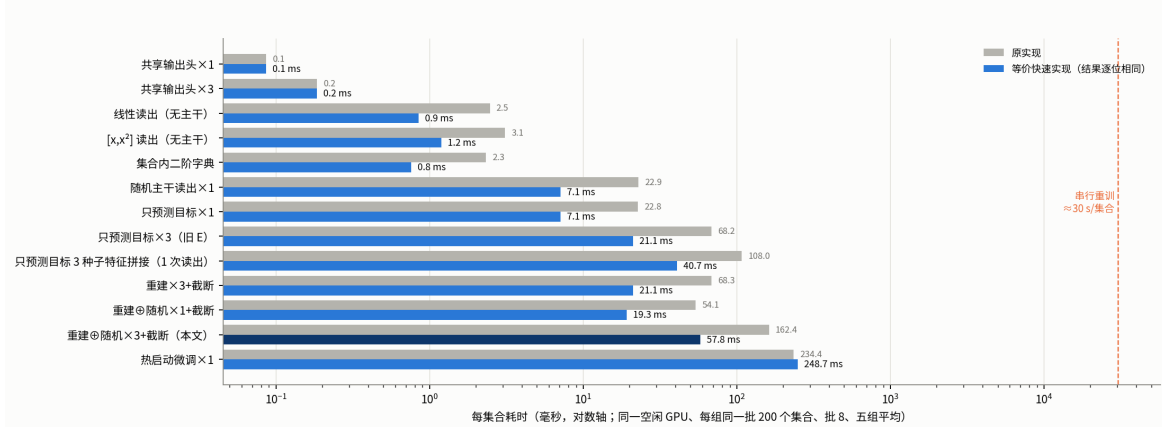


图 1: 126 号：各方法每集合耗时（灰：原实现；蓝：等价快速实现；红色虚线为串行重训约 30 秒 / 集合）

原图：DNN_Aggresvation126/figures/fig1_time_per_method.png

等价快速实现：原实现在 5 折 $\times$ 6 个 λ 中，每折都重新计算 Gram 矩阵，每个 λ 单独求解。改为全体训练行的 $Z^T Z$ 、 $Z^T y$ 只算一次，各折用“全体减去验证折”得到，6 个 λ 合并为一次批量求解。只改计算顺序，与原实现、与 125 号保存的估计值逐位相同，读出快约 2.8 倍。

图2 耗时—精度：共享输出头最快但 M 误差约为本文的 4 倍；单种子拼接 (C2) 耗时 1/3、平均精度接近本文但随种子波动；本文 (C3) 在精度上最好

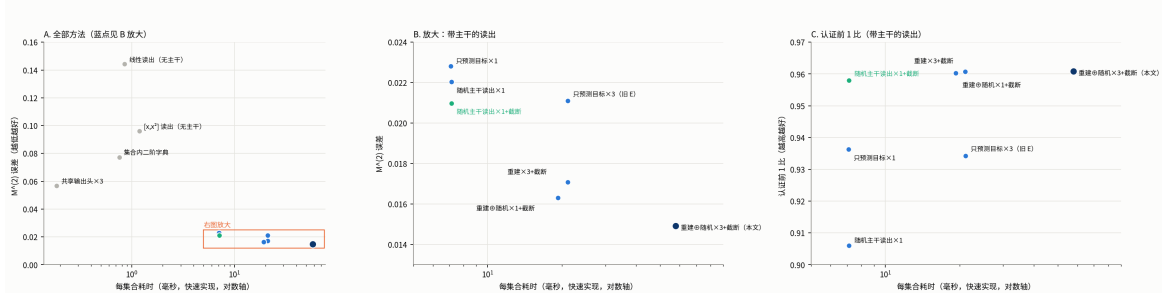


图 2: 126 号：耗时与精度的关系（左：全部方法；中：带主干的读出放大；右：认证前 1 比）

原图：DNN_Aggresvation126/figures/fig2_time_vs_accuracy.png

表 2: 126 号：代表方法的耗时与精度（快速实现，五组数据）

方法	每集合耗时	$M^{(2)}$ 误差	认证前 1
共享输出头 $\times 3$ （不做逐集合读出）	0.2 ms	0.057	0.883
随机主干读出 $\times 1$ + 截断	5.7—8.0 ms	0.021	0.958
旧 E	16.9—23.5 ms	0.021	0.934
本文 C3	48.4—63.9 ms	0.015	0.961
热启动微调 $\times 1$ （100 步）	227—299 ms	V 误差 10/10 个目标更差	—
串行重训（三攻击器）	14—56 s	精确	精确

- 读出耗时基本由特征维度决定：约 300 维 6—8 ms，约 570 维 16—21 ms。
- 最省事的共享输出头快两个数量级，但 M 误差约为本文的 4 倍。
- **随机主干 + 截断**是很便宜的备选：7 ms， M 误差与旧 E 相当，但仍比本文差 40%。
- 扫描一张完整的 $K = 2$ 表，本文需要 1—7 分钟，串行重训需要 7—53 小时。一次性预训练每个重建主干约 35—46 秒。

2 与外部“免重训”方法对比 (127 号)

2.1 选了哪些方法

论文中估计器的对比分成两块：**设计选择**（第 5 次讲的各种变体）和**与外部通用方法的对比**（本节）。外部方法都用默认设置、按原文实现，不做额外调优。

表 3: 127 号：外部 baseline

类别	方法	出处
全模型不重训	被移除字段以均值代替 (Dropout)	Williamson 等；Sun & Raskutti
全模型线性化	LazyVI（参数处线性化 + 岭修正）	Gao 等，ICML 2022
全模型热启动	热启动 + 早停	Sun & Raskutti, AISTATS 2025
掩码代理模型	随机掩码网络直接输出	Covert 等，JMLR 2021；FastSHAP
表格基础模型	TabPFN v2（上下文推断，不训练）	Hollmann 等，Nature 2025

2.2 结果

图1 与外部免重训方法的对比：只有 TabPFN 与本文精度相当，但每集合慢 13-110 倍；其余方法 M 误差为本文的 3-16 倍

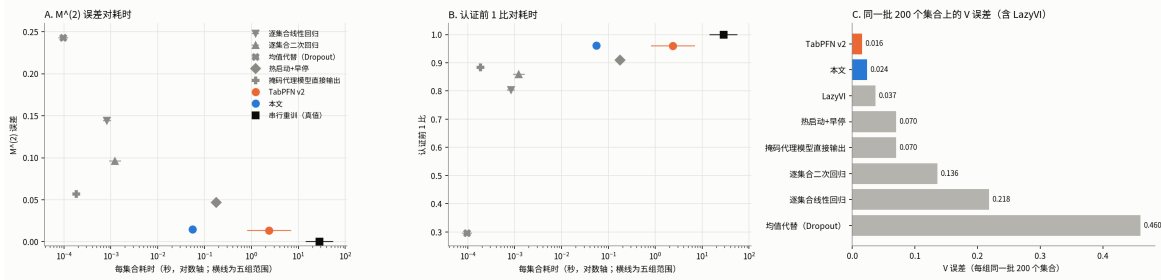


图 3: 127 号：与外部方法的对比（左： $M^{(2)}$ 误差对耗时；中：认证前 1 比对耗时；右：同一批 200 个集合上的 V 误差）

原图：DNN_Aggresvation127/figures/fig1_baselines.png

表 4: 127 号：10 个目标平均 (LazyVI 太慢，只在 200 个集合上算了 V 误差 0.037)

方法	$M^{(2)}$ 误差	排序 Spearman	认证前 1	关键召回 $\tau = 0.5 / 0.7$	每集合耗时
串行重训 (真值)	0	1	1	1 / 1	14—56 s
逐集合线性回归	0.144	0.737	0.803	0.24 / 0.16	0.7—1.0 ms
均值代替 (Dropout)	0.243	0.534	0.295	0.03 / 0.00	0.1 ms
LazyVI (抽样 200 集合)	—	—	—	—	1.5—9.1 s
热启动 + 早停	0.047	0.932	0.910	0.67 / 0.47	176—181 ms
掩码代理模型直接输出	0.057	0.930	0.883	0.55 / 0.47	0.2 ms
TabPFN v2	0.013	0.979	0.959	0.88 / 1.00	0.8—7.0 s
本文	0.015	0.968	0.961	0.91 / 0.79	48—64 ms

- 均值代替在小集合上完全失效：用全部字段训练的网络，无法只靠三个字段做预测。
- LazyVI 精度尚可，但需要双精度，每集合 1.5—9.1 秒，全表约 25 GPU 小时，所以只跑了 200 个集合的样本。
- 掩码代理模型直接输出最快，但 M 误差约为本文的 4 倍。这再次说明，逐集合闭式读出是让掩码主干变准的关键。
- TabPFN 是唯一与本文精度相当的方法。

2.3 本文与 TabPFN 分数据组看

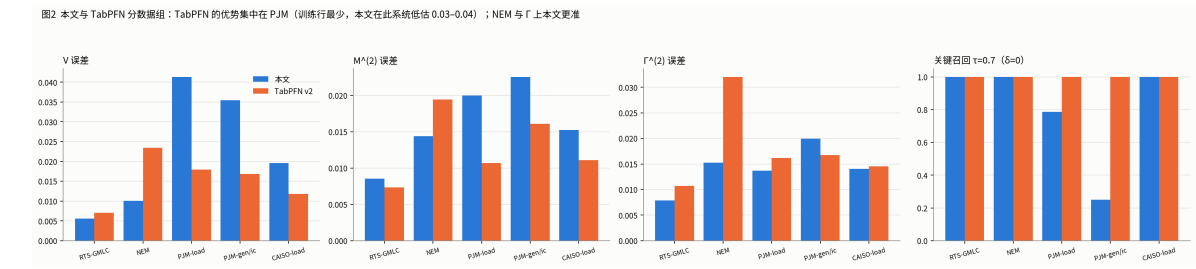


图 4: 127 号：本文与 TabPFN 分数据组对比

原图: DNN_Aggesvation127/figures/fig2_ours_vs_tabpfn.png

表 5: 127 号：分数据组 (本文 / TabPFN)

数据	V 偏差	$M^{(2)}$ 误差	$\Gamma^{(2)}$ 误差	关键召回 $\tau = 0.7$
RTS-GMLC	-0.003 / +0.004	0.0085 / 0.0074	0.0079 / 0.0107	1 / 1
NEM	0.000 / -0.018	0.0144 / 0.0195	0.0153 / 0.0320	1 / 1
PJM 负荷	-0.038 / +0.015	0.0200 / 0.0107	0.0137 / 0.0162	0.79 / 1
PJM 发电 / 联络线	-0.032 / +0.016	0.0226 / 0.0161	0.0200 / 0.0167	0.25 / 1
CAISO 负荷	-0.016 / +0.008	0.0153 / 0.0111	0.0141 / 0.0146	1 / 1

- TabPFN 的优势集中在 **PJM** (训练行最少, 本文在此有 0.03—0.04 的系统低估, PJM 发电 / 联络线在 $\tau = 0.7$ 下的召回只有 0.25); 在 **NEM** 上本文明显更好。
- TabPFN 的平均偏差为正, 说明它**有时比重训的攻击器族更强**。论文据此在局限中说明: 报告的风险对拥有此类模型的攻击者只是下界, 并建议把它作为攻击器族的一员。
- **认证之后两者没有差别**: 认证前 1 为 0.959 对 0.961, 前 3 为 0.979 对 0.978。

为什么速度差距是决定性的: 两者**摊销的对象不同**。TabPFN 在不同数据集之间摊销学习, 每个字段集合对它都是一个新任务, 要逐个目标送入 8×12 层 transformer; 本文在同一数据集的所有字段集合之间摊销, 每次查询只需几次小网络前向和三次线性求解 (所有目标共用)。审计需要成千上万次查询: NEM 的 $K = 2$ 全表, TabPFN 要 2.7 小时, 本文 5 分钟; CAISO 规模 4 的 52,360 个集合, TabPFN 要 12 小时, 本文 54 分钟。

3 论文当前版本

表 6: 论文版本

版本	内容
V5.1_en / V5.2_en	估计器换成 C3、下游全部重算；估计器一节拆成“设计选择”与“外部对比”，加入 127 号的 baseline 表和图
V6_ieee	IEEE 双栏版（21 页，含 4 个附录），在 V5.2 基础上做全文文字修订
IJCIP / JISA 定向稿	以 V6 为母版的两个投稿备选版本：IJCIP 以关键基础设施保护、物理机制为主线；JISA 以背景预算泄露画像及其验证为主线。尚未提交

4 小结

小结

- 本文估计器每集合 48—64 ms，比串行重训快 241—906 倍；快速读出与原实现逐位相同。
- 外部方法中只有 TabPFN 精度相当，但慢 13—110 倍；认证后两者相同。其余方法的 M 误差为本文的 3—16 倍。
- 需要写进论文的局限：训练行少的数据集上本文会系统低估；更强的攻击器（如 TabPFN）存在时，报告的风险是下界。